Tecnológico de Monterrey
Escuela de Ingeniería y Ciencias

Monterrey Campus

Autonomy of Unmanned Aerial Vehicles

MR3003C.601

Report:

Practice 09

State Feedback Gain Matrix for Rotary Inverted Pendulum

Part B. Regulator Control Design

Name:

Carlos Hernán Auquilla Larriva | A00834778

Teachers:

Carlos Gustavo Sotelo Molina, Ph.D

David Alejandro Sotelo Molina, Ph.D

Due date:

October 8th, 2024

Table of contents

1. Desired Poles based on the Performance Indexes 3

Function to Compute the Desired Poles based on the Desired Performance Indexes 3

Script 3

2. State Space representation of the Rotary Pendulum model in Inverted Configuration 3

Matlab Sript to obtain the State Space representation of the System 3

Controllability Verification 4

3. State Feedback Gain Matrix obtained with the Companion Method 4

Function to obtain the K gain matrix with the Companion Method 4

Main Script and comparison with other Methods 5

4. System Response 5

Performance Indexes 6

5. Conclusion 7

1. **Desired Poles based on the Performance Indexes**

Function to Compute the Desired Poles based on the Desired Performance Indexes

```
function poles = calculatePoles(overshoot, settling_time)
%Compute the second order desired poles
%Based on the desired performance indexes of overshoot and settling time
%the function computes the desired second order closed-loop poles

    % Convert overshoot to damping ratio (zeta)
    zeta = -log(overshoot) / sqrt(pi^2 + log(overshoot)^2);

    % Calculate natural frequency (omega_n) using settling time
    w_n = 4 / (zeta * settling_time);

    % Calculate the poles of the second-order system
    sigma = -zeta * w_n;
    imag_part = w_n * sqrt(1 - zeta^2);

    % Return the poles as a complex number pair
    poles = [sigma + 1i * imag_part, sigma - 1i * imag_part];
end
```

Script

```
%Compute desired poles
MP=6.81/100;%Overshoot
tss=1.54;%sec
desPoles_2nd = calculatePoles(MP,tss);%Calculate 2nd order desired poles
extra_poles=[-40, -45];%Non-dominant poles
P = [desPoles_2nd, extra_poles];
```

```
P =

  -2.5974 + 3.0371i  -2.5974 - 3.0371i -40.0000 + 0.0000i -45.0000 + 0.0000i
```

2. **State Space representation of the Rotary Pendulum model in Inverted Configuration**

Matlab Sript to obtain the State Space representation of the System

```
%% Step 1: Load Qube Servo 3 Parameters
clear;clc;close all;
```

```
pendParam; %load pendulum parameters
%Pendulum configuration
d = -1; %1 for crane configuration, -1 for inverted configuration

%%State Space Representation matrices
A = (1/Jt)*[0       0           Jt                    0;
            0       0            0                   Jt;
            0 mp^2*l^2*r*g -Jp*(br+km^2/Rm)     d*mp*l*r*bp;
            0 -d*mp*g*l*Jr d*mp*l*r*(br+km^2/Rm) -Jp*bp;];

B = [   0;
        0;
    km*Jp/(Rm*Jt);
    -d*mp*r*l*km/(Rm*Jt);];

C = [1 0 0 0;
     0 1 0 0];

D = [0;0];

sys = ss(A,B,C,D);
```

Controllability Verification

```
%Compute controllability matrix
CO = ctrb(A,B);
if rank(CO)==length(A)
    disp("The system is controllable");
else
    disp("The system is not controllable");
end
```

```
The system is controllable
```

3. State Feedback Gain Matrix obtained with the Companion Method

Function to obtain the K gain matrix with the Companion Method

```
function [K_comp] = companionM(A,B,P)
%Compute Fulll state feedback K matrix with the Companion Matrix Method
%   K = K_tilde/W

%Convert to Controllable Canonical Form
n=length(A);
A_tilde = zeros(n);
for i=1:n-1
    A_tilde(i,i+1) = 1;
```

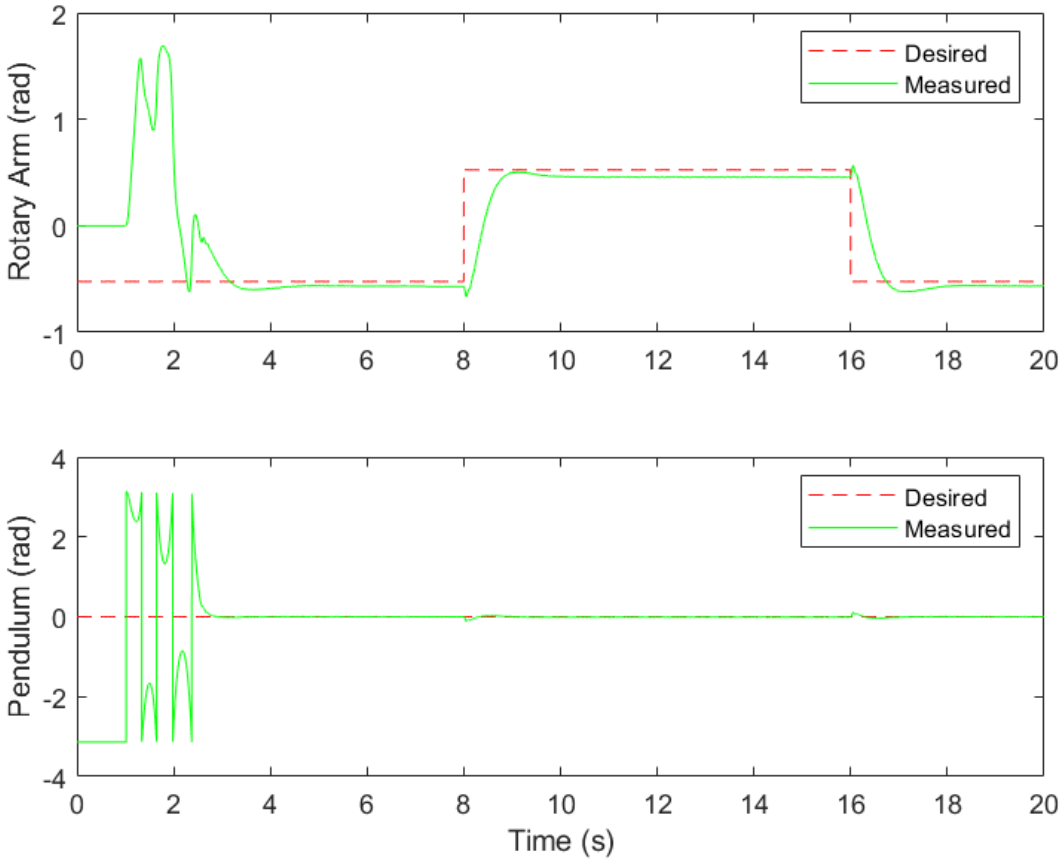
```
end
coeff = flip(poly(A));
A_tilde(n,:) =  -coeff(1:n);
B_tilde = zeros(n,1);
B_tilde(end) = 1;

K_tilde = place(A_tilde,B_tilde,P);
T_tilde = ctrb(A_tilde,B_tilde);
T=ctrb(A,B);
W_tilde = T/T_tilde; %%check this
K_comp = K_tilde/W_tilde;

end
```

Main Script and comparison with other Methods

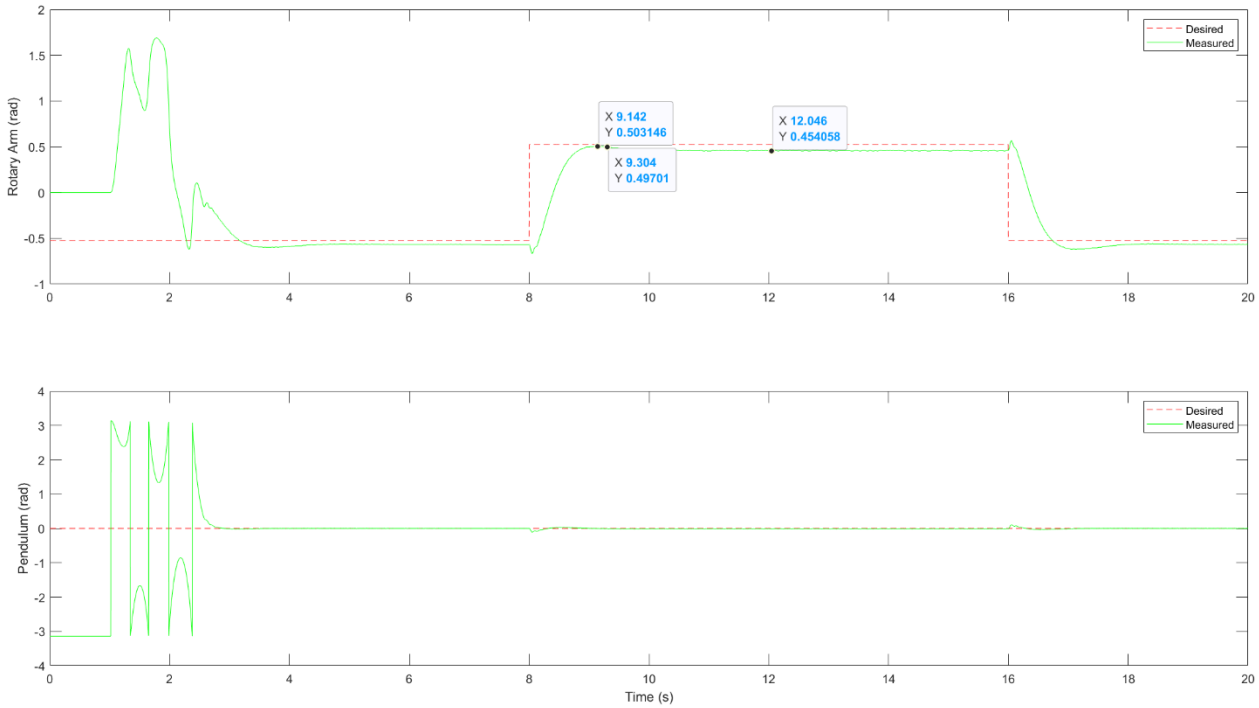
```
%Compute the state feedback matrix K
%Matlab Method
K = place(A,B,P)
%Transformation Matrix T Method
[K_transf,~,~] = transformT(A,B,P)

%Companion Matrix Method
k_comp = companionM(A,B,P)
```

```
K =

   -4.4225   49.2701   -1.8678    3.2597


K_transf =

[-4.4224922466873117811585597876501, 49.270090452263943148934822662889, -1.8677540948120417344131145663465, 3.2596630881490283088377907257527]


k_comp =

   -4.4225   49.2701   -1.8678    3.2597
```

4. System Response



Performance Indexes




```
MP =

    0.1081


settlingTime =

    1.3040
```

5. Conclusion

In this lab, we designed and implemented a state feedback controller for the rotary inverted pendulum. The process followed a structured approach, which involved several key steps:

1. Model Development: We began by developing the state-space representation of the rotary inverted pendulum in its inverted configuration. This allowed us to describe the system dynamics and prepare it for control design.

2. Check for Controllability: Controllability was assessed by the computation of the controllability matrix. This ensured that the system was controllable, and thus a state feedback controller could be effectively implemented.

3. Computation of K Using Multiple Methods: The state feedback gain matrix K was computed using several methods, including the Companion Matrix Method, Direct Substitution, and Transformation Matrix methods. Each method provided insight into how the poles could be placed to meet the system's performance criteria.

4. Steady-State Error: During the experiment, we analyzed the steady-state error of the system, ensuring that the controller minimized this error. While the state feedback controller showed good performance in maintaining the pendulum's balance, the steady-state error in the arm position tracking was observed.

5. Overshoot Requirement Not Achieved: Although the system was stabilized and the desired pole locations were achieved, the overshoot requirement was not fully met. This discrepancy between the theoretical model and actual performance could be attributed to unmodeled system dynamics, actuator limitations, or external disturbances. Further fine-

tuning of the controller parameters, possibly incorporating an integral component, could help mitigate this issue.

In summary, the designed controller was effective in stabilizing the rotary inverted pendulum, though certain performance metrics, such as the overshoot, were not fully satisfied. The experiment reinforced the importance of rigorous system modeling, stability analysis, and the use of various control design techniques to achieve the desired system behavior. Future improvements could focus on reducing overshoot and further minimizing steady-state errors.